

JAVA INTERVIEW PREPARATION

CHAPTER 22

JMM, SYNCHRONIZED, AND VOLATILE

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

The Java Memory Model, synchronized, and volatile: Reasoning About Shared State

Senior / Lead Java Developer Reading Chapter

Concurrency bugs are difficult because source-code order is not the whole execution model. Compilers optimize, processors reorder and cache memory operations, and threads interleave at many legal points. The Java Memory Model, usually called the JMM, defines which values a thread is allowed to observe and which synchronization actions create reliable ordering between threads.

The JMM does not describe a particular CPU cache. It is a language-level contract that lets the JVM optimize while giving correctly synchronized programs portable behavior.

Three Separate Problems

Shared-memory correctness requires reasoning about atomicity, visibility, and ordering.

atomicity	- is an operation observed as one indivisible action?
visibility	- when does one thread observe another thread's write?
ordering	- which actions are guaranteed to occur before other actions?

These are related but not interchangeable. A write can be atomic yet not promptly visible. A value can be visible while a compound check-and-update remains racy.

```
if (stock > 0) {  
    stock--; // read, decision, and write are not one atomic transaction  
}
```

Two threads can both read one item in stock and both decrement it. Making the field visible does not protect the invariant.

Data Races and Happens-Before

A data race exists when different threads access the same variable, at least one access is a write, and the accesses are not ordered by a happens-before relationship. The result may be legal under the JMM but surprising to a developer reasoning only from wall-clock time.

Happens-before is a visibility and ordering guarantee. If action A happens-before action B, the effects of A are visible to B, and A is ordered before B for JMM reasoning.

Important relationships include:

- Program order within one thread.
- Unlocking a monitor happens-before a later successful lock of that monitor.
- A volatile write happens-before a later volatile read of the same variable.
- Actions before Thread.start happen-before actions in the started thread.
- All actions in a thread happen-before another thread successfully returns from join.
- Completing a Future establishes the documented visibility relationship with a successful get.
- Relationships are transitive.

Thread A	Thread B
write configuration	
unlock monitor	-----HB--> lock same monitor
	read configuration safely

Happens-before is not necessarily chronological tracing language. It is the formal ordering that makes communication reliable.

What synchronized Guarantees

Every Java object can act as a monitor. Entering a synchronized block acquires that monitor; leaving releases it, including when an exception exits the block.

```
final class Inventory {
    private int available;

    synchronized boolean reserveOne() {
        if (available == 0) return false;
        available--;
        return true;
    }
}
```

The monitor provides mutual exclusion: only one thread executes code guarded by the same monitor at a time. It also provides visibility: writes made before monitor release are visible after a later acquisition of that monitor.

The phrase "same monitor" is essential. These methods do not coordinate:

```
synchronized (lockA) { state++; }
synchronized (lockB) { System.out.println(state); }
```

Both sides of an invariant must follow one locking protocol.

Instance and Static Synchronization

An instance synchronized method locks `this`. A static synchronized method locks the Class object. They are different monitors.

```
synchronized void updateInstance() { } // locks this
static synchronized void updateGlobal() { } // locks Example.class
```

Two different instances can execute their instance synchronized methods simultaneously. If state is static but guarded only by instance locks, the protection is incomplete.

Prefer a private final lock when exposing the object monitor would allow unrelated external code to participate in the locking protocol.

```
private final Object stateLock = new Object();
```

Keep Critical Sections Cohesive and Small

A critical section should cover the complete invariant but avoid unrelated slow work. Protecting only the final write is often too narrow; holding the lock across database or network I/O is usually too broad.

```
Reservation reserve(String id) {
    Reservation reservation;
    synchronized (stateLock) {
        reservation = allocateLocally(id);
    }
    auditClient.send(reservation); // remote call outside the lock
    return reservation;
}
```

Moving I/O outside a lock changes failure semantics, so the design may require an outbox, compensation, or state transition. Lock optimization must preserve the business transaction, not merely reduce waiting.

Java monitors are reentrant: a thread holding a monitor may enter another synchronized method guarded by the same monitor. Reentrancy prevents self-deadlock in nested calls, but deeply implicit locking can still be hard to maintain.

wait, notify, and the Condition Predicate

`wait` releases the monitor and suspends the thread. The thread must reacquire the monitor before returning. `notify` wakes one waiter; `notifyAll` wakes all current waiters. All must be called while owning that monitor.

Always wait in a loop because wakeups can be spurious and another thread may consume the condition first.

```
synchronized Message take() throws InterruptedException {
    while (queue.isEmpty()) {
        wait();
    }
    return queue.removeFirst();
}
```

Prefer higher-level utilities such as `BlockingQueue`, `CountDownLatch`, or `Condition` for most application code. Understanding monitor waiting remains important for interviews and legacy diagnostics.

volatile: Visibility Without Mutual Exclusion

A volatile write publishes the written value and earlier actions to a thread that later reads that same volatile variable. Volatile also restricts reorderings around the access.

```
final class ServiceLoop implements Runnable {
    private volatile boolean running = true;

    public void stop() {
        running = false;
    }

    public void run() {
        while (running) {
            performOneUnit();
        }
    }
}
```

Without a visibility mechanism, the worker is not guaranteed to observe the update as intended. Volatile is appropriate here because correctness depends on the latest independent flag value, not a compound relationship among several fields.

Why volatile Does Not Make Increment Atomic

Increment is a read-modify-write sequence.

Thread A reads 5	Thread B reads 5
Thread A writes 6	Thread B writes 6
expected 7, observed 6	

```
private volatile int count;
void increment() {
    count++; // still racy
}
```

Use a synchronized invariant, AtomicInteger, LongAdder for suitable statistics, or another ownership model. Volatile protects communication about a value; it does not combine several actions into one atomic update.

Safe Publication

An object should not become reachable by other threads before construction is complete. Safe publication can occur through a static initializer, volatile reference, lock, concurrent collection, thread-start relationship, or properly constructed immutable object with final fields.

```
final class Configuration {
    private final URI endpoint;
    private final int retries;

    Configuration(URI endpoint, int retries) {
        this.endpoint = endpoint;
        this.retries = retries;
    }
}
```

Final fields receive special initialization guarantees when `this` does not escape during construction. Final does not make referenced mutable objects immutable. A final List reference can still refer to a list that another thread mutates.

Avoid registering listeners, starting threads, or placing `this` into shared storage from a constructor. Those actions can expose a partially initialized object.

Double-Checked Locking

Lazy singleton initialization using double-checked locking requires a volatile reference.

```
private static volatile Client instance;

static Client instance() {
    Client result = instance;
    if (result == null) {
        synchronized (ClientHolder.class) {
            result = instance;
            if (result == null) {
                result = new Client();
                instance = result;
            }
        }
    }
    return result;
}
```

Volatile prevents publication from being observed independently of the initialization actions. In ordinary code, an enum singleton, eager initialization, dependency-injection container, or initialization-on-demand holder is often simpler.

Immutable Snapshots and Ownership

The easiest shared mutable state is the state that is not shared or not mutable. A common design builds a new immutable snapshot and swaps one volatile reference.

```
private volatile RoutingTable routing = RoutingTable.empty();

void refresh() {
    RoutingTable next = loader.loadImmutableTable();
    routing = next;
}

Route route(Request request) {
    return routing.match(request);
}
```

Readers need no lock because each snapshot is complete and immutable. The volatile write publishes it. This works when updates can replace the whole logical state and temporary old snapshots are acceptable.

Thread confinement is another strong option: one thread or actor owns mutable state, and other threads communicate by messages. This converts memory races into message-ordering and capacity questions.

Production Perspective

Typical failures include volatile counters losing updates, different locks guarding the same invariant, unsafe publication of mutable configuration, blocking I/O inside a hot monitor, and swallowing `InterruptedException` in wait loops. CPU profiles may show monitor contention; thread dumps show `BLOCKED` threads and lock owners; Java Flight Recorder can reveal contention duration.

Do not add volatile randomly until a test passes. Document the invariant, identify every access, choose one communication mechanism, and verify that all participating code follows it. Race bugs that disappear under logging may be timing-sensitive rather than fixed.

Interview-Ready Summary

The JMM defines legal observations and synchronization ordering. Correctness requires separate reasoning about atomicity, visibility, and ordering. Happens-before relationships are established by monitors, volatile access, thread lifecycle actions, futures, and other specified synchronization points.

`Synchronized` provides mutual exclusion and visibility for code using the same monitor. `Volatile` provides visibility and ordering for a variable but does not make compound operations such as `increment atomic`. Safe publication prevents threads from observing partially initialized state. Prefer immutability, snapshots, and ownership when they reduce shared mutation.

Revision Notes

- The JMM is a language contract, not a diagram of one CPU cache.
- Atomicity, visibility, and ordering are separate concerns.
- A data race lacks a happens-before ordering between conflicting accesses.
- Unlocking a monitor happens-before a later lock of that monitor.
- A volatile write happens-before a later read of the same volatile variable.
- `Thread.start` and `Thread.join` create visibility relationships.
- `synchronized` combines mutual exclusion with visibility.
- Synchronization works only when accesses use the same monitor protocol.
- Instance `synchronized` methods lock this; static ones lock the `Class` object.
- Monitors are reentrant.

- Keep the complete invariant inside the critical section.
- Avoid slow remote I/O while holding locks when semantics allow.
- Always test a wait condition in a loop.
- Volatile is suitable for independent flags and published references.
- Volatile increment is still a non-atomic read-modify-write.
- Publish objects only after construction completes.
- Final-field guarantees require that this does not escape construction.
- Double-checked locking requires a volatile reference.
- Immutable snapshots plus a volatile reference can remove reader locking.
- Diagnose contention and ownership rather than adding synchronization blindly.